SHM-EM: A forecast-aware event management framework for heterogeneous engineering monitoring

Ji'an Liao[a,b], Zifa Wang[a,b,*], Dengke Zhao[a,b], Jianming Wang[a,b], Zhaoyan Li[a,b], Siran Yang[a,b]

[a] Key Laboratory of Earthquake Engineering and Engineering Vibration, Institute of Engineering Mechanics, China Earthquake Administration, Harbin 150080, China

[b] Key Laboratory Earthquake Disaster Mitigation, Ministry of Emergency Management, Harbin 150080, China

[*] Corresponding author: Zifa Wang, Zifa@iem.ac.cn. Institute of Engineering Mechanics, China Earthquake Administration, No. 29 Xuefu Road, Harbin 150080, China.

Abstract

Engineering-monitoring software often separates data acquisition, time-series forecasting, and event response. This separation binds model inputs to project-specific schemas, obscures temporal and engineering semantics across forecasts, and makes it difficult to admit predictions safely into formal event workflows. SHM-EM is an open-source framework that treats persisted forecasts as contextualized and auditable inputs to engineering-event management. It contributes three software mechanisms: a versioned engineering-semantic data-model contract; a synchronized Project Future State that aggregates multi-target forecasts on one project timeline; and a controlled forecast-to-event transition that separates audited candidate evaluation without formal business side effects from gated Execute. The revised validation uses a de-identified excavation reference case, a synthetic bridge configuration used solely as a software-reuse fixture, a 15-case validation matrix comprising one positive control, 12 failure-path cases, and two input-availability controls, repeated runtime measurements, and a concrete event provenance trace. The reference workflow integrates six fixed-version point-forecast model bundles, 124 target channels, 40 future steps, and 4,960 persisted forecast rows. A Docker/Linux execution reproduced the logical end-to-end workflow, although its normalized prediction-output hash differed from the exact Windows reference and no numerical tolerance was applied. SHM-EM provides an auditable software workflow rather than a new forecasting algorithm or a claim of predictive generalization.

Keywords: engineering monitoring; structural health monitoring; time-series forecasting; event management; provenance; reproducible research software

Metadata

Nr	Code metadata description	Metadata
C1	Current code version	v1.0.1
C2	Permanent link to code/repository used for this code version	https://github.com/lja666/SHM-EM/releases/tag/v1.0.1 (fixed commit: d7cba1419145e6c75fe69ad63172af5f5abe5028)
C3	Legal code license	MIT License
C4	Code versioning system	Git
C5	Languages, tools, and services	Java 8; SQL; TypeScript; Python 3.10; Vue 3; Spring Boot 2.6.13; MyBatis 2.2.2; MySQL 8.4; PyTorch 2.11.0; Vite; ECharts; PowerShell 7; Docker Compose; OpenAPI
C6	Compilation requirements, operating environments, and dependencies	Back end: Java 8 and Maven 3.8+; database: MySQL 8.0+; front end: Node.js 20+ and npm; forecasting runtime: Python 3.10 with locked dependencies. Windows 10/11 with PowerShell 7 is the exact-output reference. The exercised

Nr	Code metadata description	Metadata
		Docker Compose Linux path reproduces component checks and the logical six-model-to-provenance workflow, but not a bitwise-identical normalized prediction-output hash.
C7	Developer documentation/manual	https://github.com/lja666/SHM-EM/tree/v1.0.1/docs (README, installation, reproduction, models, database, API, security, and data-availability documentation)
C8	Support email	nlfdzlja@163.com

1. Motivation and significance

The principal software challenge in engineering monitoring is fragmentation rather than a lack of sensors or forecasting methods. Structural-health-monitoring deployments combine displacement, settlement, strain, pressure, vibration, and environmental measurements [1]. Networked acquisition allows a project to use devices with different sampling and communication characteristics [2,3], while machine learning supports anomaly and damage detection [4-6]. Monitoring applications nevertheless remain tied to vendor tables, field names, units, and point identifiers, whereas model scripts frequently assume fixed columns and implicit ordering. These dependencies impede model integration, interpretation, verification, and reuse.

Existing standards and software address important but different parts of this problem. Sensor Web Enablement and the OGC SensorThings API provide standardized sensor, observation, and Web-query concepts [7,8]. Generic complex-event processing (CEP) provides established stream, window, rule, and event-generation paradigms [9]. SHM-EM does not claim that generic CEP is unable to implement additional controls, and its current release makes no SensorThings conformance or compatibility claim. A future adapter could map SensorThings observations into the SHM-EM registry, but that adapter is not implemented in v1.0.1.

Related research software is likewise complementary. GMFAgent organizes knowledge, data, and tools for ground-motion-field estimation [10]. Predictive-SHM provides multi-source ingestion, a unified logical data model, metadata-driven sensor and model registration, model adapters, pluggable forecasting, standardized timestamped forecasts, visualization, and residual- or threshold-oriented alerts [11]. SHM-EM is not a replacement for Predictive-SHM; it formalizes the downstream software boundary through which persisted forecasts become auditable inputs to formal engineering-event workflows. In particular, the Predictive-SHM primary source does not explicitly report a common multi-model prediction origin or a project-level synchronized future timeline.

Table 1 compares documented software responsibilities rather than ranking products. Third-party entries use *Yes*, *Partial*, *Not reported*, or *Not applicable*; *Not reported* means that the cited primary source did not explicitly document the capability, not that the software cannot provide it.

Table 1. Source-grounded responsibility comparison.

Capability	OGC SensorThings	Generic CEP	Predictive-SHM	SHM-EM
Heterogeneous observation access	Yes	Partial	Yes	Yes
Standardized observation semantics	Yes	Not applicable	Yes	Partial

Capability	OGC SensorThings	Generic CEP	Predictive-SHM	SHM-EM
Internal observation-to-model mapping	Not applicable	Not applicable	Yes	Yes
Model-specific ordered input contract	Not applicable	Not applicable	Partial	Yes
Pluggable forecasting/model adapter	Not applicable	Not applicable	Yes	Yes
Artifact and input-schema hash validation	Not applicable	Not applicable	Not reported	Yes
Shared prediction origin and future timeline	Not applicable	Not applicable	Not reported	Yes
Project-level future-state aggregation	Not applicable	Not applicable	Not reported	Yes
Rule/event evaluation	Not applicable	Yes	Partial	Yes
Candidate evaluation without formal business side effects	Not applicable	Not reported	Not reported	Yes
Execution-time eligibility recheck	Not applicable	Not reported	Not reported	Yes
Formal event-to-prediction provenance link	Not applicable	Not reported	Not reported	Yes

Recurrent, convolutional, and Transformer-based methods are widely used for multi-step forecasting [12-19]. SHM-EM does not compare forecasting algorithms because it does not introduce a forecasting method. It examines how fixed-version models can be registered, executed, checked, and used in event management. Model cards, data documentation, FAIR principles, software citation, and provenance standards guide publication, reuse, intended use, and auditability [20-25].

The software contributions are deliberately limited to three mechanisms:

1. **A versioned engineering-semantic data-model contract** that binds registered observations, ordered model inputs, target channels, units, transformations, model artifacts, temporal settings, and integrity hashes.
2. **A synchronized Project Future State** that summarizes target-, station-, and project-level forecast risk on one validated future timeline while keeping observed and forecast risk distinct.
3. **A controlled forecast-to-event transition** in which Evaluate retains an audit run but creates no formal event, execution Gate, response, notification, report, evidence, or prediction-link records; Execute reloads persisted forecasts, rechecks eligibility and integrity, validates rule semantics, and creates formal business and provenance records only after all required checks pass. Persisted-result integrity revalidation is a safeguard within this third mechanism, not a separate contribution.

In this paper, *forecast-aware* means that persisted forecasts are aligned, inspected, and conditionally admitted to a formal event workflow rather than used only for visualization. Fig. 1 retains the submitted three-part overview of research gaps, the SHM-EM software boundary, and the user workflow.

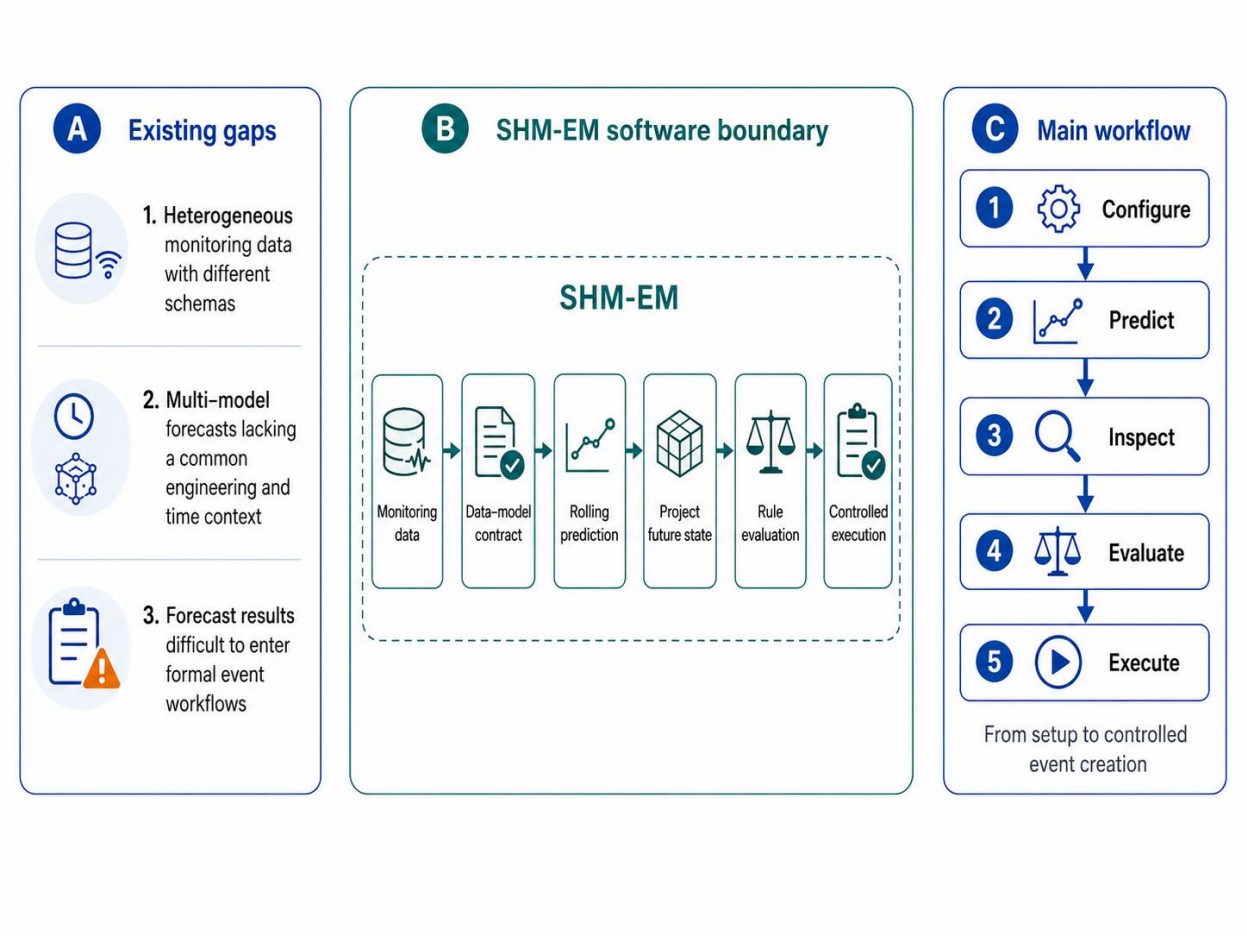

Fig. 1. Research gaps, the SHM-EM software boundary, and the forecast-aware user workflow.

1.1 Intended users and experimental setup

SHM-EM is intended for engineering-monitoring researchers, data analysts, and event-response personnel. Users configure projects, stations, instruments, and metrics, and then bind logical engineering objects to approved physical sources through the observation registry. A versioned database contract declares model artifacts, preprocessors, ordered features, target channels, units, conversion policies, temporal settings, and hashes.

A typical workflow runs the Python forecasting component PIT_PRE to persist a prediction batch. Users then inspect model runs, batch completeness, Project Future State, and joint observed/forecast series before selecting Evaluate or controlled Execute. The public reference workflow is reproduced through database scripts, APIs, tests, and PowerShell; the Docker Compose path exercises the same logical chain in Linux containers. Saved front-end state is not part of the reproduction authority.

2. Software description

2.1 Software architecture

Fig. 2 organizes SHM-EM into four layers. The presentation layer uses Vue for project, observation, prediction, rule, event, and response views. Spring Boot application services provide the observation registry, engineering conversion, joint metric series, prediction execution Gate, Project Future State, rule evaluation/execution, event response, and provenance APIs. PIT_PRE is a Python process that loads the database-authoritative model

contract, constructs aligned input windows, executes immutable model bundles, converts outputs to engineering quantities, and persists prediction batches and runs. MySQL stores observations, registry mappings, model contracts, predictions, rules, events, responses, and audit records.

The separation reflects three implementation choices. First, Python modelling dependencies do not enter the Java decision services. Second, inference is independent of page access and front-end refresh frequency. Third, formal event creation remains a backend-controlled side-effect boundary. MySQL is the implemented and validated persistence backend. The observation registry and service interfaces describe an extension boundary for other approved adapters, but no alternative time-series database has been implemented or validated.

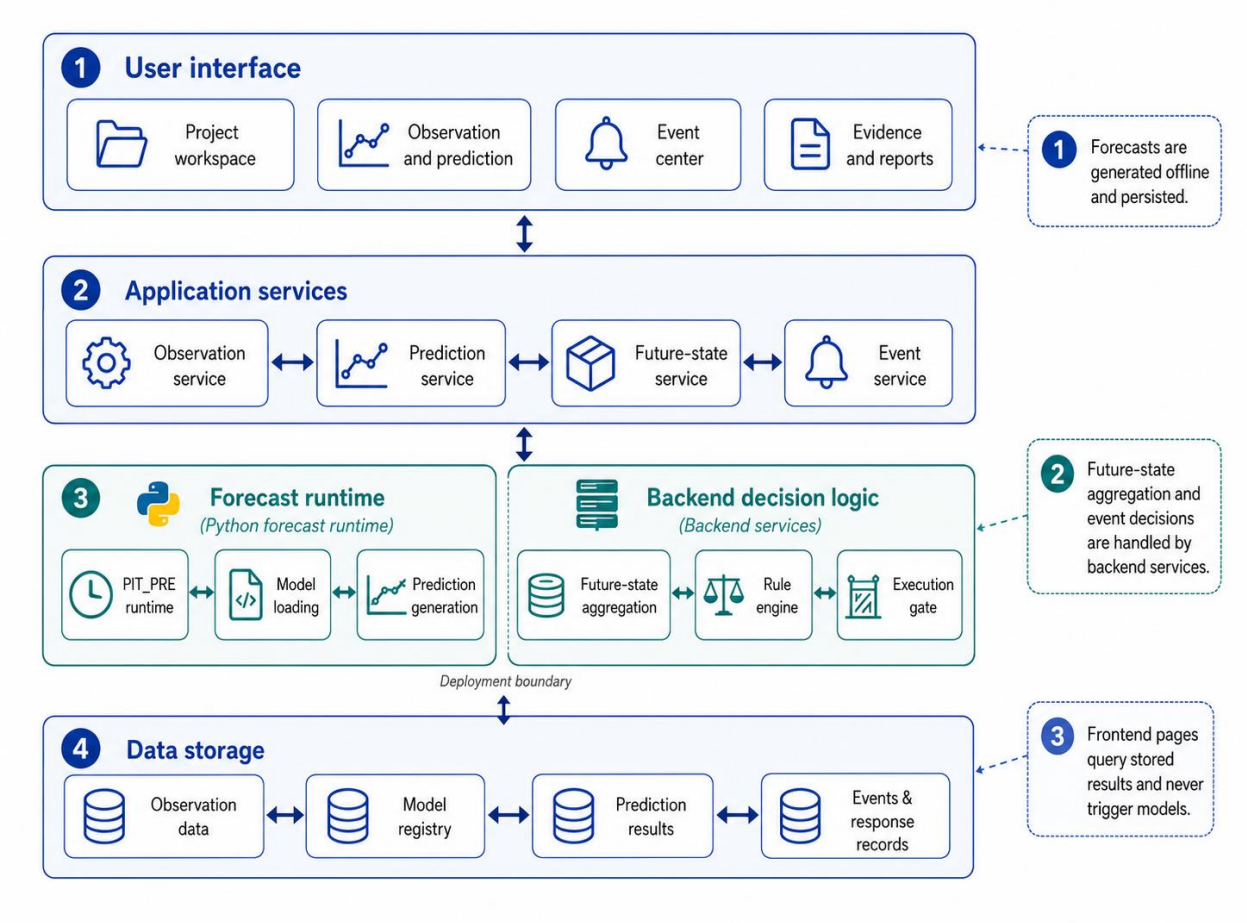

Fig. 2. Four-layer SHM-EM architecture. MySQL is the validated reference persistence implementation; the observation registry and service interfaces define the storage-adapter extension boundary.

2.1.1 Engineering monitoring object model and observation registry

A stable object model provides the shared reference for observations, forecasts, rules, and events. Projects define engineering contexts; stations represent field points, locations, or components; instruments record device and sampling properties; metrics define physical meaning, raw and engineering units, risk direction, and conversion semantics. The logical relationship is:

```
project -> station -> instrument -> metric -> timestamped observation
```

An observation carries source identity, collection time, raw value, engineering value, unit, quality, conversion operator/version/status, and conversion-parameter snapshot. Approved registry entries map logical requests to

physical storage. Front-end and API clients select registry and metric codes and do not submit physical table names. The current low-frequency adapters route four typed `em_obs_*` tables, while two explicitly registered acceleration partitions are retained for high-frequency samples. Registration does not make an arbitrary physical table valid: its columns, time semantics, units, conversion behavior, and allowlisted identifier must satisfy the adapter contract.

2.1.2 Engineering-semantic data-model contract

The data-model contract binds monitoring objects, source fields, ordered training features, fixed-version model bundles, and output targets. The database is authoritative. Each active model row records identity, target type, history length, prediction cadence and horizon, maximum operational age, runtime timeout, artifact locations, and SHA-256 values. Feature mappings record the canonical feature code, immutable training-column code, global order, source registry, station, instrument, metric, value column, raw/engineering units, required and target roles, transformations, and output-conversion version. Weight, preprocessor, inference-script, optional parameter, runtime-manifest, environment, schema, and combined bundle hashes are checked before inference.

The public contract exposes a common ordered source pool of 164 features and 124 target channels across six models. The frozen preprocessors select 114 aligned columns for YD, XD, Strain, Pressure, and Water, and 164 for Settlement. These widths are distinct from database model-feature mapping counts and output-target counts. Table 2 reports the artifact- and regression-reconciled dimensions. The public input builder forms a 16-step common source window because the longest registered history is 16 steps; individual runner windows use the final 12-16 steps required by their contracts. Pressure declares a conservative 13-row runner window and its inference script consumes the final 12 rows (`m=10`, `lag=2`).

Table 2. Verified model-bundle and tensor contract.

Model	Engineering target	Required history	Aligned input features	Output targets	d_model	Heads	FF dimension	CNN channels/kernel	Parameter source
YD	Deep horizontal displacement Y (mm)	16	114	42	128	4	64	32/3	inference-script fallback
XD	Deep horizontal displacement X (mm)	12	114	42	64	8	128	64/7	best-parameter file
Strain	Earth-pressure strain (microstrain)	13	114	14	96	4	64	48/5	best-parameter file
Pressure	Earth pressure (MPa)	13	114	14	64	1	64	24/3	best-parameter file
Water	Groundwater elevation (m)	13	114	2	96	1	128	16/3	best-parameter file
Settlement	Surface settlement (mm)	12	164	10	96	8	256	48/5	best-parameter file

All six bundles implement the deployed Transformer-encoder/CNN-branch architecture described in their immutable model cards and configuration exports. The repository retains the full 164-feature contract and

complete artifact, preprocessor, script, runtime, environment, input-schema, and bundle hashes. Listing 1 shows a compact real subset rather than a synthetic schema.

Listing 1. Compact extract from the versioned contract.


```
{
  "contractVersion": "pit_pre_contract_v1",
  "featureMappingVersion": "pit_pre_v1",
  "timeline": {
    "predictionMode": "rolling",
    "expectedSteps": 40,
    "timeStepMinutes": 3,
    "horizonMinutes": 120,
    "sharedBaseTimeRequired": true
  },
  "model": {
    "code": "settlement",
    "version": "pit_pre_v1",
    "requiredHistoryRows": 12,
    "modelFeatureMappingCount": 50,
    "alignedInputFeatureCount": 164,
    "outputTargetCount": 10,
    "inputSchemaHash": "5c2f6f0f...672daa65f1"
  },
  "feature": {
    "order": 115,
    "featureCode": "dtu1_point1_settlement_value",
    "trainingFeatureCode": "1point1settlement_value",
    "sourceRegistryCode": "SHM_EM_PUBLIC_SAMPLE_STATIC_LEVEL",
    "sourceMetricCode": "static_level_value_mm",
    "sourceValueColumn": "raw_value",
    "inputValueMode": "RAW",
    "rawUnit": "mm",
    "engineeringUnit": "mm",
    "required": true,
    "predictionTarget": true,
    "outputConversionOperatorCode": "static_level_reference_compensation",
    "outputConversionVersion": "static-level-v2-positive-20260713"
  }
}
```


The full example is schema-validated in `docs/revision/examples/data-model-contract.example.json`; the complete database-derived export is `artifacts/revision/manuscript/data-model-contract-export.json`.

Missing and asynchronous observations

Input construction uses an explicit deterministic policy. For each required feature, observations are matched backward-asof to the canonical 3-min grid within one cadence. Remaining partial gaps are processed by the declared linear interpolation and boundary-fill policy. Each run records the alignment stage, signed source-time offsets, interpolation and boundary-extension counts, fill ratio, and gap summaries in the input snapshot. A partial dropout can therefore be resolved when this declared policy forms a complete required window. If an entire required feature is unavailable, or any required value remains unresolved, inference is rejected. Freshness is checked separately by the execution Gate: OPERATIONAL mode uses wall-clock age, whereas REPLAY uses scenario time so that historical reproduction is not rejected merely because it is old relative to the current clock.

2.1.3 Multi-target rolling forecasts and Project Future State

A prediction batch identifies one project, one common base time, one forecast cadence and horizon, and the required model set. Every model run records its contract, artifact, input window, alignment diagnostics, and result-integrity metadata. Every result records batch/run/model/target identity, future step, horizon, timestamp, raw prediction, engineering prediction, unit, conversion operator/version/status, and quality.

The Project Future State is a deterministic software representation of synchronized predictions and their rule-bound risk summaries. It does not replace formal rules, quantify predictive uncertainty, or imply multi-physics joint learning. Algorithm 1 is derived from the fixed revised implementation.

Algorithm 1. Policy-bound Project Future State aggregation.

```
INPUT projectId, optional batchId, optional requestedHorizon,
      executionMode, optional referenceTime

1  Require the project and load its active Future State policy.
2  Validate the supported policy keys and its canonical SHA-256 hash.
3  Resolve a successful batch owned by the project.
4  Normalize the requested horizon to the positive batch horizon.
5  Inspect the execution Gate for the requested mode/reference time.
6  Load engineering-valued prediction points; discard null or failed conversions.
7  Load enabled rule levels and index them by metric code.
8  For each forecast feature, in step order:
9      maintain an independent streak for every rule level;
10     apply unit-compatible threshold/operator semantics;
11     assign UNASSESSED, NORMAL, or the highest activated severity;
12     retain the governing threshold and activation timestamp.
13 Forecast risk <- maximum assessed forecast severity.
14 Observed risk <- maximum severity of open observed events.
15 Overall risk <- policy merge of observed and forecast risk.
16 Aggregate target summaries by target type.
17 Aggregate station summaries by station and ordered contributors.
18 Aggregate the future timeline by step and earliest activated exceedance.
19 State hash <- SHA-256 of canonical batch, horizon, policy, target,
                 station, and timeline content.
20 Return observed, forecast, overall, target, station, timeline, Gate,
   policy, and state-hash fields.
```

Exact equality activates inclusive (>=, <=) but not strict (>, <) operators; `between` includes both bounds. A nonmatching step resets the feature/rule-level streak. Severity, rather than forecast magnitude alone, governs risk ordering. The activation time is the step at which the consecutive condition becomes satisfied. Gate eligibility is reported with the Project Future State but does not alter its deterministic summary and is not a hidden Execute prerequisite.

2.1.4 Controlled transition from forecasts to engineering events

Observed and predicted measurements are exposed through a common `MetricSeriesPoint` representation containing object, metric, time, raw value, engineering value, unit, quality, source type, and provenance. The prediction path adds batch, run, model, step, conversion, and integrity context. The same semantic rule engine can therefore evaluate Observation or Prediction inputs without conflating their provenance.

Validation, rule semantics, Evaluate, and Execute are distinct safeguards. Contract and persisted-integrity validation determine whether a prediction batch is structurally eligible. Rule validation checks metric/unit/operator compatibility. Evaluate performs a non-persisted REPLAY Gate inspection, returns candidate calculations, and persists one evaluation/audit run; it creates no formal event, execution Gate, response workflow/step, notification, report, evidence, or prediction-link record. Execute does not consume or trust a stored evaluation. It reloads the canonical series, recomputes and persists the Gate for the requested mode, validates the rule again, and only then creates formal records and provenance. Fig. 3 makes this order explicit.

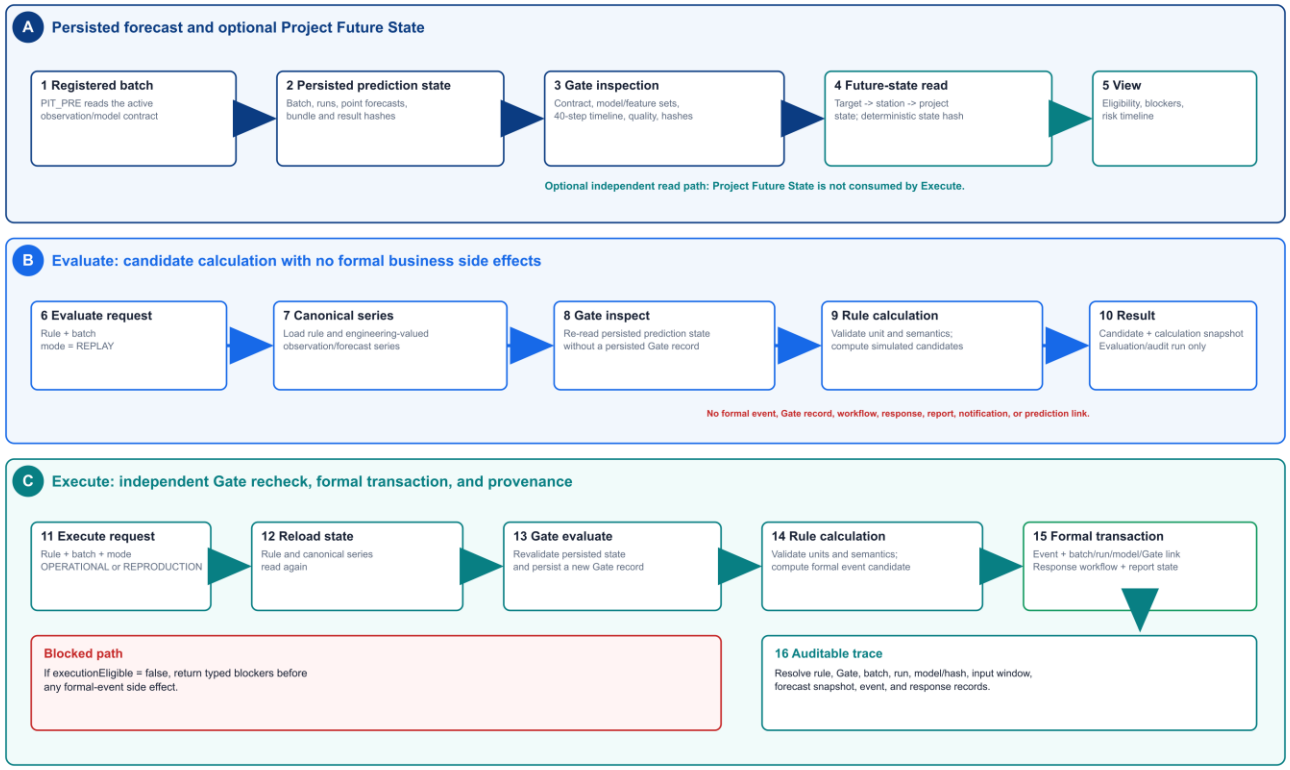

Fig. 3. Controlled sequence from persisted forecasts through optional Project Future State inspection, audited Evaluate, independently gated Execute, and formal provenance.

2.1.5 Response, provenance, and reproduction

A formal event may be linked to a response workflow, notification task, report, generic image/video attachment, and audit record. SHM-EM does not include camera control, video streaming, or automatic screenshot capture. For a forecast-driven event, `em_event_prediction_link` binds the event to the prediction batch/run, execution Gate, first exceedance, lead time, peak, consecutive steps, forecast snapshot, and result identity. Model and input hashes remain available through the linked run and evidence export.

Reproduction assets include a de-identified input window, fixed-version model bundles and preprocessors, model cards, database scripts, locked dependencies, component tests, workflow tests, PowerShell and Docker Compose entry points, and machine-readable expected results [22-24,26-30]. Environment-specific database identifiers are excluded from normalized within-run checks. Cross-platform output identity is assessed separately and is not relaxed through a numerical tolerance.

2.2 Main functionalities

SHM-EM provides four task-oriented functions:

- **Project and observation management:** configure engineering objects, register approved storage adapters, retain raw and engineering values, and expose units, quality, conversion versions, and source provenance.
- **Prediction and Project Future State:** register immutable model bundles, execute rolling forecasts, inspect batch/run completeness and Gate reasons, compare observed and predicted engineering series, and summarize target/station/project risk.
- **Rule, event, and response management:** configure versioned rules for Observation or Prediction inputs, inspect candidates through Evaluate, execute eligible rules through a fresh Gate recheck, and manage response and evidence records.

- **Reproduction and audit:** initialize the public database, run component and workflow checks, execute the six-model reference batch, and resolve a formal event back to its rule, forecast, model, input, and integrity evidence.

The revised Fig. 4 compresses three submitted screenshot pages into one compact composite. The panels are illustrative interfaces, not substitutes for the software-validation evidence in Section 3.

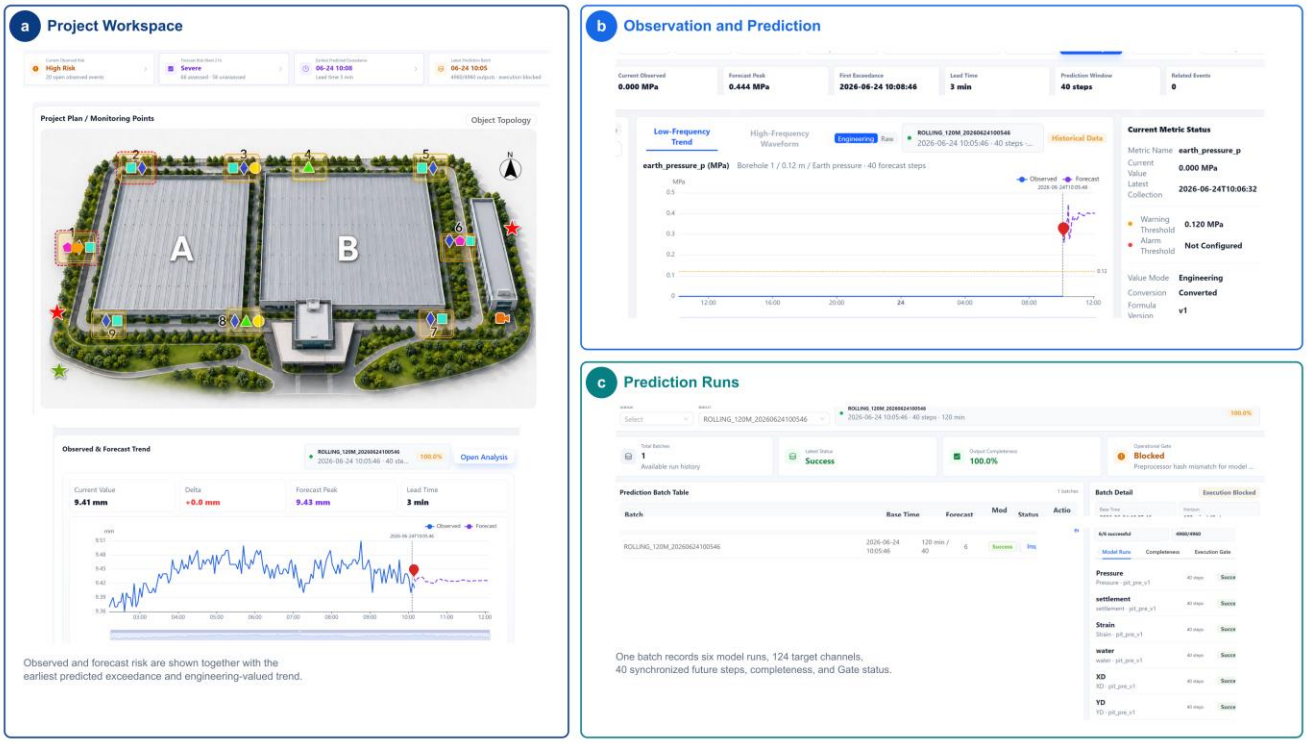

Fig. 4. Task-oriented interface views of SHM-EM: (a) project-level observed and forecast risk, (b) a joint engineering-valued observation/forecast series, and (c) prediction-batch completeness and execution eligibility. The interface is illustrative; quantitative validation is reported in the contract, failure-path, runtime, reuse, and provenance evidence.

2.3 Rule configuration and interfaces

SHM-EM exposes APIs for Project Future State queries, rule evaluation, controlled execution, and provenance. Rules are versioned database records configured through the Rules and Events interface or the API; endpoint schemas are documented in OpenAPI. A rule defines the input source (`OBSERVATION` or `PREDICTION`), metric, operator, threshold and unit, minimum consecutive steps, severity, and applicable temporal scope. Prediction rules additionally identify a batch or eligible prediction context.

The engine supports latest, maximum, minimum, mean, rate-of-change, absolute, interval, baseline-relative, and consecutive-step conditions for one metric. The public case uses threshold and consecutive-step rules. Cross-metric compound rules are outside v1.0.1. Evaluate and Execute share calculation logic but have intentionally different side-effect semantics, as described in Section 2.1.4.

3. Software validation

The validation evidence is organized by independent evidence families. Counts are not summed into one global total because backend tests, failure injections, end-to-end checks, and reproduction benchmarks overlap in behavior.

Table 3. Software testing evidence.

Test family	Cases/checks	Passed	Status
Backend unit/service/API tests	55	55	PASS
PIT_PRE contract/alignment/integrity tests	13	13	PASS
Validation matrix (P00, F01-F12, I01-I02)	15	15	PASS
Second-configuration end-to-end acceptance	7	7	PASS
Front-end typecheck and production build	2	2	PASS
Public reference end-to-end reproduction	1	1	PASS

No code-coverage percentage is reported because a stable coverage instrument is not part of the submitted release.

3.1 Public excavation-monitoring reference case

`SHM_EM_PUBLIC_SAMPLE` is derived from an operational excavation project and contains the minimum de-identified, time-shifted window required for reproduction. It contains 2,464 low-frequency observations from nine field monitoring points and covers deep horizontal displacement, earth pressure, groundwater level, and hydrostatic level. Original coordinates, device identifiers, project location, and operational records are removed, and no formal events are preloaded. Sixteen synchronized 3-min steps form the common source window; model-specific histories use the final 12-16 steps. Vibration is not part of the public forecast workflow, and the conceptual site plan is schematic and not to scale.

Engineering data-use agreements and confidentiality restrictions prevent release of the complete training data, raw historical series, and point-level train/validation/test splits. The public model cards report data categories and ranges, split principles, preprocessing, aggregate validation metrics, dimensions, hashes, and limitations. The public sample is an inference and software-reproduction window; it is not an independent accuracy or cross-project generalization test.

The reference batch completed six model runs, 124 target channels, 40 future steps at 3-min intervals, and 4,960 persisted forecast rows. Contract, conversion, referential integrity, Gate, Project Future State, Evaluate, Execute, response, and provenance checks completed under the reference workflow.

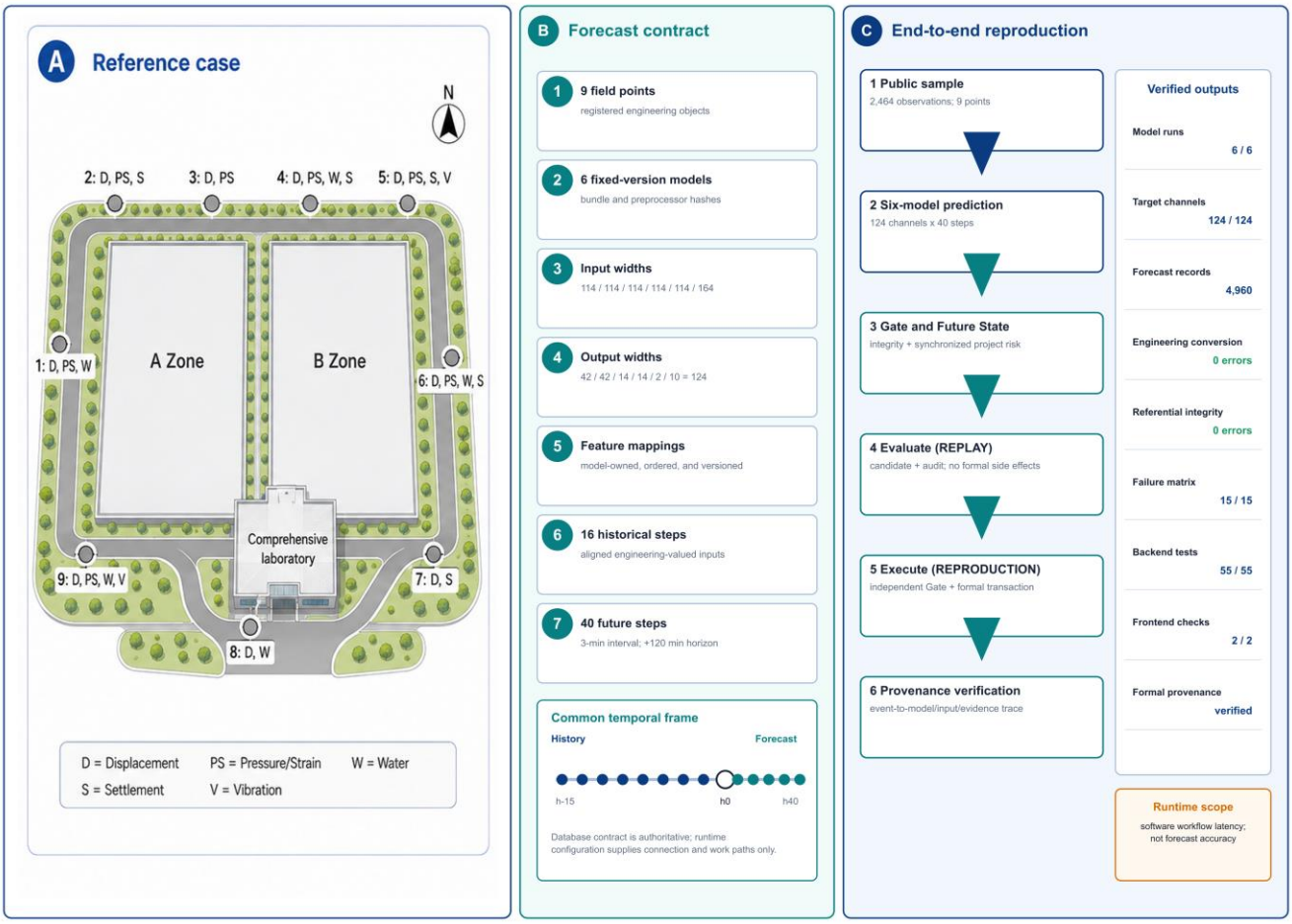

Fig. 5. Public reference case, verified six-model contract, common temporal frame, and end-to-end reproduction checks.

3.2 Cross-configuration reuse

Cross-configuration reuse was evaluated with one independently registered synthetic bridge-monitoring fixture. The fixture exercises configuration, data-model, prediction, Gate, Future State, rule, event, response, provenance, API, and front-end boundaries. It is not a bridge forecasting study.

Table 4. Synthetic bridge software-reuse fixture.

Registered or produced item	Count/result
Project	1
Stations	3
Instruments	12
Metric bindings	26
Observation mappings	4
Feature mappings	164
Compatible model bundles	2
Rules	1
Persisted forecast rows	1,120

Registered or produced item	Count/result
End-to-end functional checks	7/7 PASS
Frozen backend source modifications	0
Frozen front-end workflow source modifications	0
PIT_PRE core modifications	0
Existing observation-table schema alterations	0

Two registered compatible model bundles, used solely as software-workflow fixtures, produced 1,120 forecast rows. Persisted-result integrity and execution eligibility passed; Project Future State was assessed; Evaluate retained its audit run but produced no formal business side effects; Execute created one formal event, one response workflow, four response steps, and one prediction-provenance link; an unavailable required mapping was rejected before inference; and the existing project routes, joint-series API, and front-end build remained usable. Their outputs are not interpreted as bridge-domain predictive validation or cross-domain forecasting accuracy. The result demonstrates configuration reuse for this tested fixture only; it does not establish universal no-code onboarding or arbitrary model compatibility.

3.3 Failure-path and execution-safety validation

A 15-case validation matrix comprising one positive control, 12 failure-path cases, and two input-availability controls was executed in isolated reproduction databases. Table 5 groups the cases by boundary. All cases expected to be blocked produced zero formal event, response-workflow, response-step, report, evidence, or prediction-link side effects.

Table 5. Validation matrix and side-effect boundary.

Group	Cases	Condition	Expected/observed boundary
Positive control	P00	Valid reference prediction and rule	Execute succeeds; one formal event is created
Contract/model/batch	F01, F03, F04, F06, F08, F11	Incomplete steps; artifact hash mismatch; missing target; schema mismatch; failed run; failed batch	Execution Gate blocks
Timeline/quality/integrity	F02, F07, F09, F10	Stale prediction; temporal misalignment; corrupted persisted values with stale hashes; invalid quality	Gate or persisted-result integrity blocks
Rule semantics	F05	Incompatible engineering unit	Rule validation blocks
Evaluate-to-Execute mutation	F12	Persisted state is changed after Evaluate	Execute reload/recheck blocks
Input availability	I01, I02	Partial dropout; entire required feature unavailable	Declared alignment policy resolves the partial gap; unavailable required feature rejects input assembly

F09 exposed the need to recompute persisted-result integrity rather than trust stored hash metadata. The narrowly scoped correction revalidates the canonical persisted rows before formal execution. F12 demonstrates that a prior candidate does not authorize later mutation: Execute reloads the batch and performs an independent Gate and rule check. The matrix supports the tested failure boundaries; it is not an absolute safety certification.

3.4 Runtime and bounded scalability

Runtime was measured on the documented Windows reference environment using the fixed revised implementation. Repeated measurements report median and p95; one-off MySQL characterization is listed separately. The workloads were single-process research-reproduction runs, not multi-user production-throughput tests.

Table 6. Selected runtime and bounded-scaling evidence (ms).

Operation	Workload	n	Median	p95	Single elapsed
Full six-model prediction batch	6 models, 124 targets, 40 steps	30	16,778.359	18,729.326	-
Input assembly	Same reference batch	30	5,632.700	6,260.991	-
All-model inference	Same reference batch	30	5,257.484	6,604.427	-
Engineering conversion	Same reference batch	30	2,951.386	3,544.938	-
Prediction persistence, exclusive estimate	4,960 rows	30	1,352.796	1,535.914	-
Persisted-integrity hashing	4,960 rows	30	531.599	693.269	-
Execution Gate inspection	4,960 rows	30	343.129	407.100	-
Project Future State	Public reference	30	472.342	574.761	-
Rule Evaluate	Public reference	30	269.465	313.340	-
Rule Execute	Public reference	10	317.238	336.361	-
Event provenance trace	Public reference	30	2.578	20.692	-
Gate stress S1	4,960 rows, 124 targets, 40 steps	10	2,406.939	2,666.804	-
Gate stress S2	49,600 rows, 1,240 targets, 40 steps	10	3,603.382	3,843.174	-
MySQL persistence S1	4,960 rows	1	-	-	16,131.595
MySQL persistence S2	49,600 rows	1	-	-	186,431.707
Independent integrity verification S1	4,960 rows	1	-	-	316.746
Independent integrity verification S2	49,600 rows	1	-	-	4,100.818

S1 and S2 characterize two bounded endpoints; they do not demonstrate linear scalability or an O(N) relation. The Gate currently inspects at most 50,000 prediction-display rows. This is an application-level bounded-query safeguard, not a measured MySQL capacity limit. MySQL is the only implemented backend, and no conclusion is drawn about a TimescaleDB, InfluxDB, or other time-series-native adapter.

3.5 Provenance and reproducibility

One formal reproduction event was traced from its public identifier to the rule, prediction, model, input, Gate, forecast, and response records. Table 7 presents the human-readable chain; the complete 40-step series and persisted-integrity fields are retained in `artifacts/revision/manuscript/provenance-trace-final.json`.

Table 7. Concrete forecast-event provenance trace.

Trace element	Captured value
Formal event	FEVT-4-f61b7667dcc01721aa2a
Rule	PRED_GROUND_SETTLEMENT_WARNING, version v2
Prediction batch	ROLLING_120M_20250101004202_RUN_20260830232819008787, batch ID 40
Base time	2025-01-01 00:42:02
Model run	Run 236; settlement version pit_pre_v1
Model artifact SHA-256	3c18be8ae8fcdb1f8c740e8d0bf1c3e8775a5c0d1d11994d4360be1213c7ad40
Input window	2024-12-31 23:57:02 to 2025-01-01 00:42:02
Input-schema SHA-256	5c2f6f0f2351b15675fc223b36043729b1e7f8ab0bd08caa891593672daa65f1
First activated exceedance	2025-01-01 00:45:02; lead time 3 min; peak 9.43204345 mm; two consecutive steps
Gate	Gate 1; eligible; persisted-result integrity independently revalidated
Formal side effects	One event, one response workflow, four response steps, one report, and one prediction link

The event-trace API exposes the event, rule-linked batch/run/model/input-window metadata, artifact and input-schema hashes, forecast snapshot, and Gate identity. The public export obtains persisted run/batch hashes from the isolated database because the API does not expose every persisted integrity field directly. After export, the reproduction script restores all append-only formal tables to their recorded baseline.

Windows 10/11 with PowerShell 7 remains the exact-output reference. The exercised Docker/Linux path completed component checks and the logical six-model -> 4,960 results -> Gate -> Project Future State -> Evaluate -> Execute -> provenance workflow. Input and model-contract hashes matched, and all 4,960 rows matched structurally by target and step. However, the normalized prediction-output hash differed (`exactPredictionReproduction=false`), the maximum persisted absolute difference was 0.00285349, and no tolerance was applied (`toleranceApplied=false`). The full row-wise comparison artifact is retained. Native Ubuntu-host execution was not separately captured.

4. Impact

4.1 Reproducible and auditable forecast integration

SHM-EM makes a model run inspectable as a software artifact rather than an implicit script invocation. The versioned contract identifies ordered features, engineering objects, units, conversion versions, temporal settings, artifacts, preprocessors, scripts, runtime dependencies, and hashes. The public reference combines these records with a deterministic input window and expected machine-readable outputs. The software evidence includes 55 backend tests, 13 PIT_PRE tests, the 15-case validation matrix, seven second-configuration checks, two front-

end checks, and one public end-to-end reproduction. These overlapping families are reported separately rather than summed.

This evidence establishes contract checking, workflow reproduction, and traceability for the released configurations. It does not establish forecasting superiority, calibrated uncertainty, production throughput, or reliability improvement relative to an unspecified conventional system.

4.2 Cross-configuration reuse

In the synthetic second-configuration experiment, one bridge fixture was registered with three stations, 12 instruments, 26 metric bindings, four observation mappings, 164 feature mappings, two compatible workflow-fixture model bundles, and one rule. The fixed revised release completed seven end-to-end functional checks and produced 1,120 forecast rows without modifying the fixed backend business source, front-end workflow source, PIT_PRE core, or existing observation-table schemas.

This is direct evidence of configuration reuse for one heterogeneous software fixture. It does not validate bridge-domain forecast accuracy, cross-domain model transfer, arbitrary source-table support, universal no-code onboarding, or compatibility with model bundles that do not satisfy the declared adapter and contract requirements.

4.3 Controlled event transition and traceability

The controlled transition separates exploratory candidate inspection from formal engineering records. Evaluate uses the same engineering-valued series and rule semantics as Execute and retains an evaluation/audit run, but creates no formal event, Gate, response, notification, report, evidence, or prediction-link record. Execute reloads persisted forecasts, revalidates artifact/contract/timeline/quality/freshness and persisted-result integrity, checks engineering units, and creates event, response, and provenance records only after all checks pass. The failure matrix shows zero formal side effects for every case expected to be blocked, and the concrete provenance trace demonstrates how one successful event resolves to its rule, batch, model artifact, input window, forecast snapshot, Gate, and response workflow.

This mechanism supports audit and reproducible inspection. SHA-256 detects accidental corruption, stale hash metadata, and uncoordinated mutation; it is not tamper-proof against an actor able to change both records and hashes.

4.4 Current deployment and scientific scope

The current release has the following boundaries:

- The six public model bundles emit point forecasts. Gate eligibility is data/artifact/timeline/quality/freshness eligibility, not predictive uncertainty, calibrated confidence, or probabilistic risk.
- The 50,000-row Gate inspection cap is an application safeguard, not a MySQL capacity limit. MySQL 8 is the only implemented and validated persistence backend; no alternative time-series database adapter has been tested.
- The observation registry is an internal abstraction. SHM-EM has no OGC SensorThings endpoint, adapter, or conformance result.
- The software is a research reference implementation without application-level authentication. Production deployment requires TLS, an identity provider, role-based authorization with separate Execute privilege, least-privilege database access, protected audit/provenance storage, secret management, network segmentation, rate limits, and tested backup/restore procedures.

- Persisted hashes are integrity checks, not cryptographic protection against a privileged attacker. Stronger deployment requires privilege separation and an externally protected append-only store, keyed HMAC, or digital signature.
- Docker/Linux demonstrated functional and logical portability but not bitwise numerical identity. The normalized output hash differed from Windows, no tolerance was applied, and a native Ubuntu-host result was not separately captured.
- The synthetic bridge configuration is a software-workflow fixture, not external field validation or cross-domain predictive validation.
- The current adapter scope covers registered and approved observation sources. Arbitrary forecasting frameworks and physical tables may require explicit adapter and contract work.
- Forecasts must not be the sole basis for automated safety decisions; engineering review remains necessary.

5. Conclusions

SHM-EM integrates heterogeneous engineering observations, fixed-version point-forecast models, a synchronized Project Future State, versioned rules, formal events, responses, and provenance. Its three contributions are a versioned engineering-semantic data-model contract, a deterministic synchronized Project Future State, and a controlled forecast-to-event transition. The revised evidence formalizes these mechanisms with a real contract example and model configuration, a code-derived aggregation algorithm and sequence, one public excavation reproduction, one synthetic software-reuse fixture, a 15-case validation matrix, repeated runtime measurements, a concrete event trace, and a bounded Docker/Linux portability result.

The evidence supports reproducible and auditable workflow integration for the tested configurations. It does not establish predictive generalization, calibrated uncertainty, universal no-code reuse, production security, storage-backend neutrality, or exact cross-platform numerical reproduction. Future work may evaluate uncertainty-aware forecasts, paginated or chunked Gate validation, explicit alternative-storage and SensorThings adapters, stronger deployment security, compound rules, native Linux execution, and independent field configurations without weakening the current fail-closed contract and provenance boundaries.

6. Data and software availability

SHM-EM v1.0.1 is available as an immutable release at https://github.com/lja666/SHM-EM/releases/tag/v1.0.1, fixed release commit `d7cba1419145e6c75fe69ad63172af5f5abe5028`. The release archive `SHM-EM-v1.0.1.zip` has SHA-256 `ea0973b7c82e06c3c8910ec36fcf2c3d47765a87d11552337a86c69de41a7cef`; the matching checksum sidecar is published with the archive. The source code and six fixed-version model bundles are licensed under the MIT License. The public de-identified sample and conceptual site plan are licensed under CC BY 4.0. The public sample supports engineering conversion, six-model inference, prediction gating, Project Future State construction, rule evaluation, controlled formal execution, response creation, and provenance reproduction; it is not an independent model-generalization dataset.

The complete historical field data cannot be released because they include project location, original device identifiers, operational records, continuous monitoring series, and information subject to ownership and contractual restrictions. The repository documents the private-data boundary, the public subset selection, and the expected machine-readable outputs. The published archive contains the public sample, models, tests, and reproduction documentation but excludes the restricted field dataset, credentials, generated databases, local runtime state, and manuscript working files.

Acknowledgements

The authors thank the members of the research group for their contributions to software development and manuscript preparation. This work was supported by the Scientific Research Fund of the Institute of Engineering Mechanics, China Earthquake Administration (No. 2025B07), the National Natural Science Foundation of China (Nos. 52378543 and 52378544), and the National Key Research and Development Program of China (No. 2023YFC3805203).

References

[1] Farrar CR, Worden K. An introduction to structural health monitoring. Philos Trans A Math Phys Eng Sci. 2007;365(1851):303-315. https://doi.org/10.1098/rsta.2006.1928.

[2] Lynch JP, Loh KJ. A summary review of wireless sensors and sensor networks for structural health monitoring. Shock Vib Dig. 2006;38(2):91-128. https://doi.org/10.1177/0583102406061499.

[3] Hassani S, Dackermann U. A systematic review of advanced sensor technologies for non-destructive testing and structural health monitoring. Sensors. 2023;23(4):2204. https://doi.org/10.3390/s23042204.

[4] Bao Y, Li H. Machine learning paradigm for structural health monitoring. Struct Health Monit. 2021;20(4):1353-1372. https://doi.org/10.1177/1475921720972416.

[5] Azimi M, Eslamlou AD, Pekcan G. Data-driven structural health monitoring and damage detection through deep learning: state-of-the-art review. Sensors. 2020;20(10):2778. https://doi.org/10.3390/s20102778.

[6] Gomez-Cabrera A, Escamilla-Ambrosio PJ. Review of machine-learning techniques applied to structural health monitoring systems for building and bridge structures. Appl Sci. 2022;12(21):10754. https://doi.org/10.3390/app122110754.

[7] Broering A, Echterhoff J, Jirka S, Simonis I, Everding T, Stasch C, et al. New generation Sensor Web Enablement. Sensors. 2011;11(3):2652-2699. https://doi.org/10.3390/s110302652.

[8] Liang S, Khalafbeigi T, van der Schaaf H, editors. OGC SensorThings API Part 1: Sensing Version 1.1. OGC Implementation Standard 18-088. Open Geospatial Consortium; 2021. https://docs.ogc.org/is/18-088/18-088.html [accessed 1 September 2026].

[9] Cugola G, Margara A. Processing flows of information: from data stream to complex event processing. ACM Comput Surv. 2012;44(3):15. https://doi.org/10.1145/2187671.2187677.

[10] Zhao D, Wang Z, Wang J, Liao J, Li Z, Yang S. GMFAgent: domain knowledge-driven agent for ground motion field estimation. SoftwareX. 2026;34:102673. https://doi.org/10.1016/j.softx.2026.102673.

[11] Yang S, Li M, Liao J, Wang J, Zhao D, Li Z, et al. Predictive-SHM: an open-source, extensible software toolkit for multi-sensor structural health monitoring and time-series prediction. SoftwareX. 2026;35:102732. https://doi.org/10.1016/j.softx.2026.102732.

[12] Lim B, Zohren S. Time-series forecasting with deep learning: a survey. Philos Trans A Math Phys Eng Sci. 2021;379(2194):20200209. https://doi.org/10.1098/rsta.2020.0209.

[13] Kong X, Chen Z, Liu W, Ning K, Zhang L, Marier SM, et al. Deep learning for time series forecasting: a survey. Int J Mach Learn Cybern. 2025;16:5079-5112. https://doi.org/10.1007/s13042-025-02560-w.

[14] Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, et al. Attention is all you need. Adv Neural Inf Process Syst. 2017;30:5998-6008.

[15] Oreshkin BN, Carpov D, Chapados N, Bengio Y. N-BEATS: neural basis expansion analysis for interpretable time series forecasting. In: International Conference on Learning Representations; 2020.

[16] Lim B, Arik SO, Loeff N, Pfister T. Temporal Fusion Transformers for interpretable multi-horizon time series forecasting. Int J Forecast. 2021;37(4):1748-1764. https://doi.org/10.1016/j.ijforecast.2021.03.012.

[17] Zhou H, Zhang S, Peng J, Zhang S, Li J, Xiong H, et al. Informer: beyond efficient Transformer for long sequence time-series forecasting. Proc AAAI Conf Artif Intell. 2021;35(12):11106-11115. https://doi.org/10.1609/aaai.v35i12.17325.

[18] Wu H, Xu J, Wang J, Long M. Autoformer: decomposition transformers with auto-correlation for long-term series forecasting. Adv Neural Inf Process Syst. 2021;34:22419-22430.

[19] Nie Y, Nguyen NH, Sinthong P, Kalagnanam J. A time series is worth 64 words: long-term forecasting with Transformers. In: International Conference on Learning Representations; 2023.

[20] Mitchell M, Wu S, Zaldivar A, Barnes P, Vasserman L, Hutchinson B, et al. Model cards for model reporting. In: Proceedings of the Conference on Fairness, Accountability, and Transparency; 2019. p. 220-229. https://doi.org/10.1145/3287560.3287596.

[21] Gebru T, Morgenstern J, Vecchione B, Vaughan JW, Wallach H, Daume H III, et al. Datasheets for datasets. Commun ACM. 2021;64(12):86-92. https://doi.org/10.1145/3458723.

[22] Wilkinson MD, Dumontier M, Aalbersberg IJ, Appleton G, Axton M, Baak A, et al. The FAIR Guiding Principles for scientific data management and stewardship. Sci Data. 2016;3:160018. https://doi.org/10.1038/sdata.2016.18.

[23] Barker M, Chue Hong NP, Katz DS, Lamprecht AL, Martinez-Ortiz C, Psomopoulos F, et al. Introducing the FAIR Principles for research software. Sci Data. 2022;9:622. https://doi.org/10.1038/s41597-022-01710-x.

[24] Smith AM, Katz DS, Niemeyer KE; FORCE11 Software Citation Working Group. Software citation principles. PeerJ Comput Sci. 2016;2:e86. https://doi.org/10.7717/peerj-cs.86.

[25] Moreau L, Missier P, editors. PROV-DM: The PROV Data Model. W3C Recommendation. World Wide Web Consortium; 2013. https://www.w3.org/TR/2013/REC-prov-dm-20130430/ [accessed 25 July 2026].

[26] Wilson G, Aruliah DA, Brown CT, Chue Hong NP, Davis M, Guy RT, et al. Best practices for scientific computing. PLoS Biol. 2014;12(1):e1001745. https://doi.org/10.1371/journal.pbio.1001745.

[27] Sandve GK, Nekrutenko A, Taylor J, Hovig E. Ten simple rules for reproducible computational research. PLoS Comput Biol. 2013;9(10):e1003285. https://doi.org/10.1371/journal.pcbi.1003285.

[28] Pineau J, Vincent-Lamarre P, Sinha K, Lariviere V, Beygelzimer A, d'Alche-Buc F, et al. Improving reproducibility in machine learning research: a report from the NeurIPS 2019 reproducibility program. J Mach Learn Res. 2021;22(164):1-20.

[29] Lamprecht AL, Garcia L, Kuzak M, Martinez C, Arcila R, Martin Del Pico E, et al. Towards FAIR principles for research software. Data Sci. 2020;3(1):37-59. https://doi.org/10.3233/DS-190026.

[30] Stodden V, McNutt M, Bailey DH, Deelman E, Gil Y, Hanson B, et al. Enhancing reproducibility for computational methods. Science. 2016;354(6317):1240-1241. https://doi.org/10.1126/science.aah6168.